

GUIA DE CHEAT SHEET CSS

DESARROLLO DE APLICACIONES WEB -DAW135



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Introducción: ¿Qué es CSS y por qué una Cheat Sheet?

CSS (Cascading Style Sheets) es el lenguaje estándar que utilizamos para dar estilo, diseño y presentación a las páginas web. Mientras HTML define la estructura y el contenido de un documento, CSS se encarga de cómo ese contenido se ve: colores, tipografías, espaciados, animaciones y mucho más. Esta separación entre contenido y presentación es uno de los principios fundamentales del desarrollo web moderno, ya que permite mantener el código más limpio, reutilizable y fácil de mantener.

Dominar CSS puede parecer abrumador al principio, dada la enorme cantidad de propiedades, selectores, pseudoclasas y módulos que existen. Es por eso que contar con una cheat sheet —una hoja de referencia rápida— se convierte en una herramienta invaluable tanto para principiantes como para desarrolladores experimentados. En lugar de memorizar cada sintaxis, puedes consultarla en segundos mientras trabajas en tus proyectos.

Separación de responsabilidades

HTML define la estructura;
CSS controla la apariencia.
Código más limpio y
mantenible.

Reutilización de estilos

Un archivo CSS puede
aplicarse a múltiples
páginas, asegurando
consistencia visual en todo
el sitio.

Referencia instantánea

La cheat sheet te permite
consultar propiedades y
selectores sin interrumpir tu
flujo de trabajo.

Sintaxis Básica y Estructura

Toda regla CSS sigue una estructura simple pero poderosa: un **selector** que apunta a los elementos HTML, seguido de un bloque de declaraciones entre llaves. Cada declaración es un par de propiedad y valor separados por dos puntos y terminados en punto y coma. Comprender esta estructura es el primer paso para escribir CSS de forma efectiva y sin errores de sintaxis que puedan romper tu diseño.

Estructura fundamental

```
selector {  
  propiedad: valor;  
  propiedad2: valor2;  
}
```

El selector define **qué elemento** se va a estilizar.
El bloque de declaraciones define **cómo** se verá ese elemento.

Tres formas de incluir CSS

Externo (recomendado)

```
<link rel="stylesheet"  
href="style.css">
```

Interno

```
<style>  
  /* CSS aquí */  
</style>
```

En línea (inline)

```
<tag style="prop: val;">
```



Buena práctica: Siempre que sea posible, utiliza un archivo CSS externo. Esto facilita el mantenimiento, mejora el rendimiento del sitio y permite que múltiples páginas compartan los mismos estilos sin duplicar código.

Selectores Fundamentales

Los selectores son el corazón de CSS. Te permiten apuntar con precisión a los elementos HTML que deseas estilizar. Conocer la variedad de selectores disponibles y cuándo usar cada uno es esencial para escribir CSS eficiente y específico.



Selector Universal *

Selecciona absolutamente todos los elementos del documento. Útil para resets de CSS o aplicar estilos globales.

```
* { box-sizing: border-box; }
```



Selector de Tipo div, p, h1

Selecciona todos los elementos de un tipo específico en el documento. Simple y directo.

```
p { color: #333; }
```



Selector de Clase .nombre

Selecciona todos los elementos que tengan esa clase asignada. Reutilizable en múltiples elementos.

```
.destacado { font-weight: bold; }
```



Selector de ID #nombre

Selecciona un único elemento con ese ID. Alta especificidad; usar con moderación.

```
#header { background: #2D2E34; }
```

Selectores Combinadores

Combinador	Sintaxis	Descripción
Descendiente	A B	Selecciona B dentro de A (a cualquier profundidad)
Hijo directo	A > B	Selecciona B que es hijo inmediato de A
Hermano adyacente	A + B	Selecciona B inmediatamente después de A
Hermano general	A ~ B	Selecciona todos los B posteriores al mismo nivel que A

Selectores Avanzados y de Atributos

Los selectores avanzados de CSS amplían enormemente tu capacidad para apuntar a elementos específicos sin necesidad de agregar clases o IDs adicionales al HTML. Los selectores de atributos te permiten filtrar elementos por la presencia o el valor de sus atributos, mientras que las pseudoclases y pseudoelementos te dan acceso a estados dinámicos y partes específicas del contenido que no están representadas en el árbol del DOM.

Selectores de Atributos

`[attr]`

Elementos que tienen el atributo `attr`, sin importar su valor.

`[attr=value]`

Elementos cuyo atributo `attr` es exactamente igual a `value`.

`[attr^=value]`

Atributo que **comienza** con `value`.

`[attr$=value]`

Atributo que **termina** con `value`.

`[attr*=value]`

Atributo que **contiene** `value` en cualquier posición.

Pseudoclases de Estado

```
a:hover { color: blue; }  
input:focus { outline: 2px solid; }  
button:active { opacity: 0.8; }
```

Pseudoclases Estructurales

```
li:first-child { font-weight: bold; }  
li:last-child { border: none; }  
tr:nth-child(2n) { background: #eee; }  
p:not(.excluir) { color: #333; }
```

Pseudoelementos

```
.item::before { content: "→ "; }  
.item::after { content: "✓"; }  
p::first-line { font-size: 1.2em; }  
p::first-letter { font-size: 2em; }
```

Modelo de Caja (Box Model)

El Box Model es uno de los conceptos más fundamentales de CSS. Cada elemento HTML es representado como una caja rectangular compuesta por cuatro capas concéntricas: el **contenido** (content), el **relleno interior** (padding), el **borde** (border) y el **margen exterior** (margin). Comprender cómo estas capas interactúan entre sí es esencial para controlar el espacio, el tamaño y la distribución de los elementos en una página web.

Por defecto, CSS usa `box-sizing: content-box`, lo que significa que el ancho y alto definidos aplican solo al contenido, sin incluir el padding ni el border. Esto puede generar cálculos inesperados. La práctica moderna recomienda usar `box-sizing: border-box` para que el ancho total incluya padding y border, simplificando enormemente el diseño.

margin

Espacio **exterior** al borde. Separa el elemento de sus vecinos. Puede ser negativo.

```
margin: 10px 20px;  
margin: top right bottom left;
```

border

Línea **alrededor** del padding y contenido. Define el límite visual del elemento.

```
border: 2px solid #000;  
border-radius: 8px;
```

padding

Espacio **interior** entre el borde y el contenido. Expande el área clicable.

```
padding: 16px 24px;  
padding-top: 8px;
```

content

El **contenido real** del elemento: texto, imágenes u otros elementos hijos.

```
width: 300px;  
height: auto;
```



Truco esencial: Agrega `*, *::before, *::after { box-sizing: border-box; }` al inicio de tu CSS para que todos los elementos calculen su tamaño incluyendo padding y border, evitando sorpresas de layout.

Tipografía y Texto

La tipografía es uno de los elementos más poderosos del diseño web. CSS ofrece un conjunto completo de propiedades para controlar cada aspecto de cómo se muestra el texto: desde la familia tipográfica y el tamaño hasta el espaciado entre letras y la altura de línea.

Propiedades de Fuente

Propiedad	Valores comunes
font-family	"Arial", sans-serif
font-size	16px, 1.2em, 1rem
font-weight	normal, bold, 400, 700
font-style	normal, italic, oblique
font-variant	normal, small-caps

Propiedades de Texto

Propiedad	Valores comunes
text-align	left, center, right, justify
text-decoration	none, underline, line-through
text-transform	uppercase, lowercase, capitalize
line-height	1.5, 24px, normal
letter-spacing	0.05em, 2px

Unidades tipográficas clave

px — Píxeles

Unidad absoluta.
Tamaño fijo
independiente del
contexto.

em — Relativa al padre

1em = tamaño de
fuente del elemento
padre. Se acumula
en elementos
anidados.

rem — Relativa al root

1rem = tamaño de
fuente del elemento
html. Más
predecible que em.

% — Porcentaje

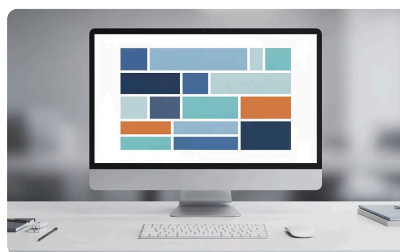
Relativo al tamaño
de fuente del padre.
Ideal para diseños
fluidos.

Importar fuentes de Google

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;700');  
  
body {  
  font-family: 'Inter', sans-serif;  
}
```

Layout: Flexbox y Grid

Flexbox y CSS Grid son los dos sistemas de layout más poderosos y modernos disponibles en CSS. Aunque ambos sirven para organizar elementos en una página, tienen filosofías distintas y se complementan entre sí. Flexbox es ideal para distribuciones en una sola dimensión —ya sea una fila o una columna—, mientras que Grid permite crear layouts bidimensionales complejos con filas y columnas simultáneamente. Conocer ambos te da el poder para construir prácticamente cualquier diseño web.



Flexbox — 1D

```
/* Contenedor */
display: flex;
flex-direction: row | column;
justify-content: flex-start | center
| space-between | space-around;
align-items: stretch | center
| flex-start | flex-end;
flex-wrap: nowrap | wrap;
gap: 16px;
/* Elementos hijos */
flex: 1; /* grow shrink basis */
align-self: center;
order: 2;
```

CSS Grid — 2D

```
/* Contenedor */
display: grid;
grid-template-columns: 1fr 2fr 1fr;
grid-template-rows: auto 200px;
grid-gap: 16px; /* o gap: 16px */
place-items: center;

/* Elementos hijos */
grid-column: 1 / 3; /* span */
grid-row: 1 / 2;
grid-area: header;
```



Regla de oro: Usa **Flexbox** para componentes (menús de navegación, tarjetas, botones alineados). Usa **Grid** para el layout general de la página (estructura de columnas, dashboards, galerías). Ambos pueden anidarse sin problema.

Colores, Fondos y Bordes

El color es fundamental para la identidad visual de cualquier sitio web. CSS ofrece múltiples formatos para definir colores, cada uno con ventajas específicas. Además del color, las propiedades de fondo y borde te permiten crear superficies ricas y visualmente atractivas, desde simples bloques de color hasta fondos con imágenes complejas y efectos de degradado.

Formatos de Color en CSS

Nombre `red`, `blue`

140 nombres de colores predefinidos.
Solo para prototipado rápido.

Hexadecimal `#FF5733`

El formato más usado en diseño. 6 dígitos hex (RGB). También acepta `#FFF` (3 dígitos).

RGB/RGBA `rgb(255, 87, 51)`

Valores de 0–255 por canal. RGBA agrega canal alfa para transparencia (0–1).

HSL/HSLA `hsl(0, 100%, 50%)`

Hue (0–360°), Saturation (%), Lightness (%). Más intuitivo para ajustar tonos.

Fondos

```
background-color: #F2F2F2;
background-image: url('bg.jpg');
background-repeat: no-repeat;
background-position: center center;
background-size: cover | contain | 100%;
background-attachment: fixed;
/* Degradado */
background: linear-gradient(
  135deg, #2D2E34 0%, #6c63ff 100%
);
```

Bordes

```
border: 2px solid #2D2E34;
border-style: solid | dashed | dotted;
border-width: 1px 2px 1px 2px;
border-color: #FF0000;
border-radius: 8px; /* redondeado */
border-radius: 50%; /* círculo */
outline: 3px dashed blue;
```

Conclusión: ¡A Practicar!

Esta cheat sheet resume lo esencial de CSS para empezar con confianza. Ahora toca dar el siguiente paso: practicar, experimentar y construir proyectos reales para afianzar lo aprendido.



Fundamentos

Practica selectores, Box Model y propiedades básicas.



Flexbox y Grid

Construye layouts completos y responsive.



Transitions y Animations

Da movimiento y vida a tus interfaces.



Diseño responsive

Adapta tu sitio con @media queries y unidades relativas.



El mejor CSS es el que se escribe. Abre tu editor, practica y construye algo increíble hoy mismo.



Recursos y Documentación

MDN Web Docs CSS

La referencia más completa de CSS.

CSS-Tricks

Guías, trucos y artículos prácticos.

W3Schools CSS

Tutoriales interactivos para principiantes.

Can I Use

Compatibilidad de propiedades CSS en navegadores.